

k -Means Clustering*Date: March 26, 2026**Eric Wong*

Acknowledgements. These notes are based on these [excellent CS 4/5780 notes](#) from Cornell, but adjusted to conform to the notation we’ve been using in this course.

1 Clustering

We’re now going to turn our attention to **clustering**, an unsupervised learning problem. In unsupervised learning, one is given some data $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ (e.g., feature vectors only). A key distinction from the supervised setting is that we no longer have corresponding labels (e.g. no y_i). The goal in unsupervised learning is just to understand some property of the data, or to find some structure or patterns in the data. Note that this is pretty vaguely defined compared to supervised learning where we had one specific objective of learning the label. This is generally going to be true throughout our brief foray into unsupervised learning over the next several lectures: the overarching goal of “finding structure” is quite vague until we discuss specific subtasks.

In clustering, the goal is to identify groups of data that are similar to each other, canonically known as “clusters”. There are generally a few reasons why one might want to do this:

1. **Data visualization.** Clusters provide a way for users to organize and visualize their data. Each cluster often be summarized with a few representative examples.
2. **Feature learning.** Clusters can provide a higher-level feature (akin to the polynomial features or other complex features) that can be used as a feature map.
3. **Recommendations.** Clustering algorithms can be used to group similar users together based on their background to provide recommendations, such as movie recommendations based on what similar users have watched.
4. **Pattern discovery.** Some clusters may be surprising, in that they group data points in a way that users did not previously expect to be similar. This can be helpful in discovering new patterns.

At a high level, the goal of k -means clustering is to partition the data into k groups where each group contains similar objects. For example, if we take similar to mean close in Euclidean distance, an ideal clustering can be found in Figure ???. In general, the exact clusters we get will depend heavily on our definition of similarity.

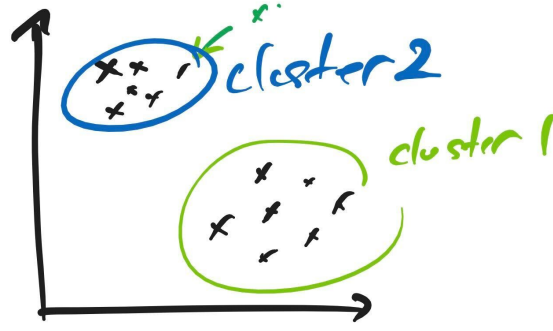


Figure 1: Figure demonstrating how data can be clustered into two clearly defined clusters based on Euclidean distance.

More formally, we assume that $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$. Our goal is to partition the data into subsets $\mathcal{C}_1, \dots, \mathcal{C}_k$ where $\mathcal{C}_j \subset \{1, \dots, n\}$. Each subset $\mathcal{C}_j$ can be viewed as the set of indices of data points that have been assigned to cluster j , i.e. $i \in \mathcal{C}_j \Leftrightarrow x_i$ is assigned to cluster j . In other words, $\mathcal{C}_j$ represents the set of data points $\{x_i : i \in \mathcal{C}_j\}$ that have been assigned to the j th cluster.

Clustering doesn't want just any partition—instead, clustering wants partitions that group similar points together. For now, let's assume that Euclidean distance represents the similarity between two points. In other words, if $\|x_i - x_{i'}\|_2$ is low, then x_i and $x_{i'}$ are similar.

What makes a good cluster, anyway? In Figure ??, it was evident what the “right” clustering was based on Euclidean distance. There were clearly two sets of points that were distinctly grouped in Euclidean space. However, in the general case, the definition of what a good cluster is may not be as obvious.

For example, Figure ?? showcases different data points with different cluster assignments for $k = 2$. While the first of these looks intuitively “bad”, it is not obvious whether the 2nd clustering is better or worse than the 3rd clustering.

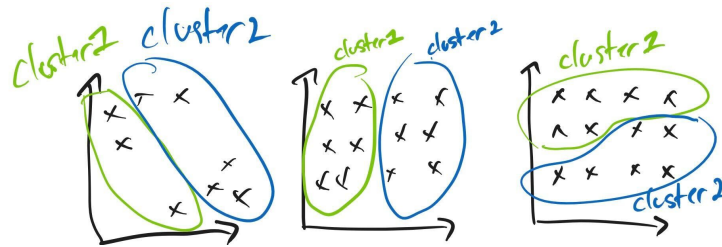


Figure 2: (Left) Examples of a bad clustering for the data in Figure ??. (Middle, Right) A different set of datapoints where the correct clustering is not as obvious.

It may also not be the case that Euclidean distance is not the right metric of similarity. For example, consider the data in Figure ??. A naive clustering for $k = 2$ based on Euclidean distance

would split this circle into two halves, when the actual structure is instead two concentric circles. A better distance would be one based on the angle and radius of each point from the center.

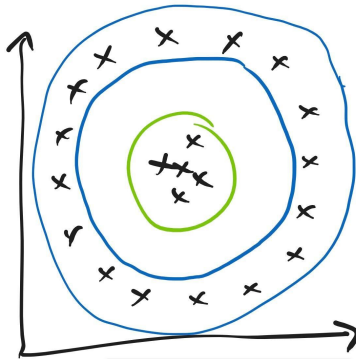


Figure 3: An example of data where Euclidean distance may not make sense for clustering.

2 k -means clustering

How do we formalize this? We would like to find a way to (a) divide our data into clusters such that (b) the examples within each cluster are most similar. This should sound very familiar: clustering can actually be formalized as a loss minimization problem that looks very similar to what we’ve done previously for supervised learning! Recall that loss minimization for supervised learning solves problem of the following form:

$$\min_{\theta} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i)$$

where we minimized the loss over parameters of our model θ . In clustering we’ll do something very similar: our “loss function” will be the total dissimilarity of each cluster $\ell(\mathcal{C}_i)$, and our “parameters” are possible partitions of the data $\mathcal{C}_1, \dots, \mathcal{C}_k$. Then, a natural way to view clustering is to minimize this dissimilarity loss over possible clusterings:

$$\min_{\mathcal{C}_1, \dots, \mathcal{C}_k} \sum_{j=1}^k \ell(\mathcal{C}_j)$$

As we will soon see, this view is not quite complete (we need a few more restrictions on the cluster assignments $\mathcal{C}_i$), but it captures the gist of how will solve this problem.

Partitions. First, we will define what we mean to partition our data into $\mathcal{C}_1, \dots, \mathcal{C}_k$. Recall that we have data $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, and that our goal is to construct k clusters $\mathcal{C}_1, \dots, \mathcal{C}_k \subseteq \{1, \dots, n\}$ where $i \in \mathcal{C}_j \Leftrightarrow x_i$ is assigned to the j th cluster. However, our clusters $\mathcal{C}_j$ cannot just be any subsets of $\{1, \dots, n\}$. We want our clusters to cover all of our data without repeating data. Formally, we represent this with the following two constraints:

1. First, we require that $\bigcup_j \mathcal{C}_j = \{1, \dots, n\}$, In other words, every data point must be assigned to a cluster.

2. Next, we also require that $\mathcal{C}_j \cap \mathcal{C}_{j'} = \emptyset$ for all $j \neq j'$. In other words, the clusters are disjoint.

In total, these requirements force our clustering to be a *partition* of the data: every point must be assigned to a cluster, and no two clusters can overlap. If we have a set of k clusters $\mathcal{C}_1, \dots, \mathcal{C}_k$ such that these two criteria are satisfied, then we have a well-defined partition of our data.

Similarity within clusters. The end goal of clustering is not only to partition the data, but to find a partition that maximizes the similarity of data within each cluster. Using the Euclidean distance as our notion of similarity¹, for a given partition we can define the loss as

$$\ell(\mathcal{C}_1, \dots, \mathcal{C}_k) = \sum_{j=1}^k \frac{1}{2|\mathcal{C}_j|} \sum_{i, i' \in \mathcal{C}_j} \|x_i - x_{i'}\|^2$$

where the smaller the loss, the better the clustering is. In other words, we are selecting a clustering that minimizes the sum of squared distances within each cluster, normalized by the size of the cluster. As it turns out, this loss can be equivalently written in terms of the centroids of each cluster:

$$\ell(\mathcal{C}_1, \dots, \mathcal{C}_k) = \sum_{j=1}^k \sum_{i \in \mathcal{C}_j} \|x_i - \mu_j\|^2$$

where $\mu_j = \frac{1}{|\mathcal{C}_j|} \sum_{i \in \mathcal{C}_j} x_i$ is the mean of the j th cluster. This formulation will closely match the eventual algorithm for k -means clustering.

Putting these two parts together, we have the final formulation that we would like to solve:

$$\begin{aligned} & \min_{\mathcal{C}_1, \dots, \mathcal{C}_k} \sum_{j=1}^k \ell(\mathcal{C}_j) \\ & \text{subject to } \bigcup_j \mathcal{C}_j = \{1, \dots, n\} \\ & \mathcal{C}_j \cap \mathcal{C}_{j'} = \emptyset \quad \forall j, j' \end{aligned}$$

If we could solve this problem, we could compute the ideal clustering that maximizes similarity within k groups. Unfortunately, this problem is pretty hard. Due to the constraints and the discrete partition variables, we can not directly apply gradient descent. In fact, to find the best clustering, we cannot do better than simply trying all k -way partitions and selecting the best one. However, we can quickly get a reasonably good solution if we give up on finding the globally optimal solution, using an approach called Lloyd's algorithm.

2.1 Lloyd's Algorithm

Lloyd's algorithm is a classic approach for clustering that uses a local search strategy. Specifically, from an initial starting point, the algorithm will iteratively update the clusters to progressively yield a better loss. In this way, we can guarantee that the loss will not degrade over time and

¹You can in principle replace Euclidean distance with any other distance metric.

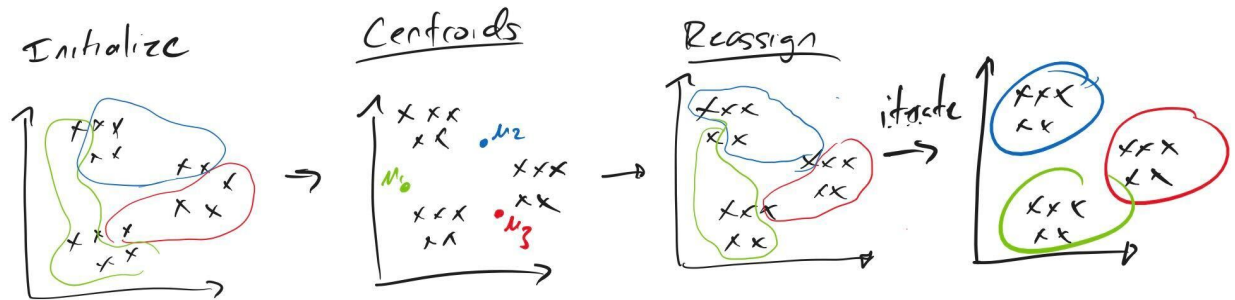


Figure 4: A depiction of Lloyd's algorithm.

converge to a local minimum. However, with this algorithm we can not guarantee that the loss will converge to the global optimum.

Lloyd's algorithm begins by initializing the the partition to some values $\mathcal{C}_1, \dots, \mathcal{C}_k$ and repeating the following steps until convergence:

1. Compute the current set of cluster means $\mu_j = \frac{1}{|\mathcal{C}_j|} \sum_{i \in \mathcal{C}_j} x_i$ for $j = 1, \dots, k$.
2. Update the cluster assignments $\mathcal{C}_1, \dots, \mathcal{C}_k$ by assigning each point x_i to its closest centroid μ_j . In other words, assign x_i to $\mathcal{C}_{j^*}$ where $j^* = \arg \min_j \|x_i - \mu_j\|^2$
3. If cluster assignments did not change, we have converged.

Figure ?? demonstrates the two main steps of Lloyd's algorithm. For n datapoints of dimension d , this algorithm has a computational cost of $O(ndk)$ per iteration. Effectively, for each data point, we must compute its distance to all k cluster centroids, an operation of cost $O(d)$. In general, there is no guarantee on the number of iterations you need to take to converge, other than it will converge to a local optimum.

Sub-optimality of clustering and k -means++. As we've mentioned previously, there is no guarantee that Lloyd's algorithm will converge to the globally optimal solution. As an example, consider the example in Figure ?. Such a cluster assignment will never move the cluster means far enough to reassign points from green to either red or blue. Indeed, a poor initial assignment of $\mathcal{C}_1, \dots, \mathcal{C}_k$ can lead to a poor solution. Since this is a local search procedure, a poor initial assignment may be unable to find the globally optimal solution.

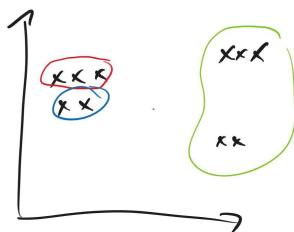


Figure 5: An example of a sub-optimal clustering.

The sensitivity of Lloyd’s algorithm to the initial assignment led to new initialization strategies, resulting in an improved version of k -means clustering called k -means++. Instead of randomly assigning points to clusters at initialization, we can instead randomly choose k cluster centers $\mu_1, \dots, \mu_k$ and then assign each point to its closest center. This initialization mirrors the second step of Lloyd’s algorithm. To pick cluster centers, we can first select μ_1 from $x_1, \dots, x_n$ uniformly at random. Then, we can pick subsequent centers $\mu_2, \dots, \mu_k$ based on a distribution weighted by distance to already selected centers. This ensures that the initial k points are sufficiently spread out.

How many clusters? A key parameter of k -means clustering is the number of clusters k . How do we pick the right k ? Suppose we naively selected a k to minimize the loss. It turns out that we can achieve a perfect clustering with zero loss by taking $k = n$ and use one cluster centroid per data point! In other words, take $\mathcal{C}_i = \{i\}$ for all $i = 1, \dots, n$. Even though this achieves zero loss, this is not a good clustering as it does not capture any new structure in the data.

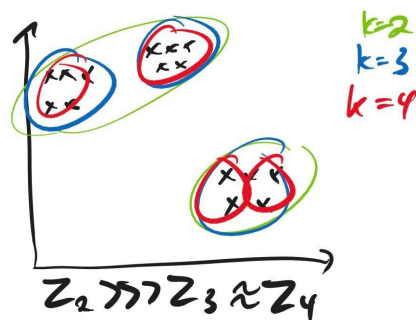


Figure 6: Examples of clusterings with different k .

However, if you have too few clusters, then you will end up merging multiple clusters together (simply by the pigeon hole principle). Therefore, we would like a way to balance between having enough clusters (i.e. large enough k) and also not having too many clusters (i.e. not too large of a k). Figure ?? depicts these scenarios of having too few or too many clusters.

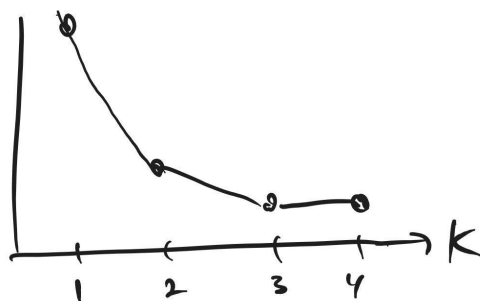


Figure 7: A typical trend of the loss as a function of the number of clusters k .

A simple heuristic is to start with a small k , and gradually increase it until the improvement in the loss stops decreasing quickly. In other words, we can plot $\ell(\mathcal{C}_1, \dots, \mathcal{C}_k)$ as a function of k and increase k to minimize the loss. However, since we know the loss will always improve, we can

stop when the marginal improvement of $\ell(\mathcal{C}_1, \dots, \mathcal{C}_k) - \ell(\mathcal{C}_1, \dots, \mathcal{C}_{k+1})$ is small. When a big cluster naturally sub-divides into two smaller clusters, this marginal improvement will be large. On the other hand, if clear cluster must be split again, the marginal improvement will be small. This is not a precise criteria but is a reasonable heuristic. Figure ?? depicts a typical plot of the loss versus the number of clusters.

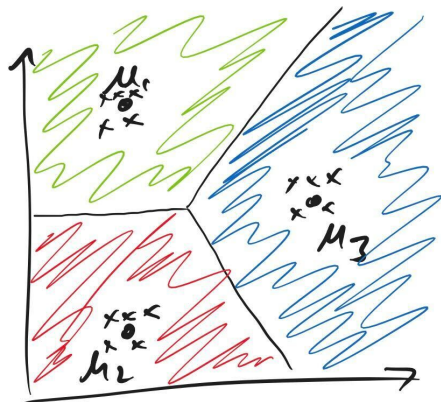


Figure 8: A typical trend of the loss as a function of the number of clusters k .

Geometric structure. We can interpret the “decision boundary” of a clustering as the regions that are assigned to a single cluster centroid. The choice of similarity score directly determines these boundaries. For example, Euclidean distance will partition $\mathbb{R}^d$ into k regions with linear decision boundaries. These boundaries look similar to what is known as Voronoi tessellations.